
Hardware/Software Codesign Proposal

Cross-Traffic Intersection Management for Autonomous Vehicles

Embedded Electronic Engineering Lab A · Week 4 Milestone

Prof. Dr. Achim Rettberg · Prof. Dr. Stefan Henkler

Team A · Group 01

Summary. This document is the Week 4 deliverable: a proposal for partitioning the Road-Side-Unit (RSU) intersection controller between software and hardware, in preparation for the Week 5 implementation in FreeRTOS and ModelSim. The proposal builds on two artefacts that already exist and agree with each other — the UPPAAL timed-automata model verified in Week 2 and the deterministic C core written in Week 3. Because both already exist, the partition is *behaviour-preserving by construction*: it changes only *where* each function runs, not the verified safety contract. The control-dominated scheduling and messaging logic stays in software; the regular, data-parallel and safety-relevant conflict checking moves into a small hardware *Conflict & Safety Coprocessor* that returns one `safe_to_grant` bit and runs an independent collision monitor. The result is a grant decision with constant, easy-to-bound timing, a second hardware check of the `A[] noCollision()` invariant, and a fairness (aging) term that removes the V5 starvation observed in Week 3.

1 Objective of Week 4

The course plan splits the project into weekly milestones. Week 3 delivered the first abstract implementation; Week 4 asks for a *proposal* for the hardware/software codesign; and Week 5 is the codesign implementation itself, with software in FreeRTOS and hardware in ModelSim. This document is the Week 4 proposal. It describes how we intend to split the RSU controller and what the interface between the two sides looks like, so that Week 5 becomes an implementation task rather than a redesign.

The proposal rests on two things we already have: the verified UPPAAL model (`RSU_MM1_MM2_priority_emergency_intersection`) and the deterministic C core. Since both already exist and produce the same behaviour, the partition does not alter the fleet, the priority order, the conflict rules, or the safety queries. It only decides the execution target of each part. The remainder of the document recaps the verified baseline, states the partitioning criterion, presents the proposed software and hardware parts together with their register interface, adds the fairness term that the Week 3 results showed we need, and closes with a verification plan, a construct-by-construct mapping, and a Week 5 work plan with risks.

2 The Verified Baseline (Weeks 1–3)

Week 1 fixed the functional requirements: the RSU must serialise conflicting movements, give priority to emergency and main-street traffic, support both M/M/1 and M/M/2 service, and communicate using only V2I, I2V and V2V messages. Week 2 turned this into a UPPAAL network of timed automata with two templates — `Vehicle` and `RSU_Controller` — instantiated as one RSU and six vehicles, and six queries were checked and all hold. Week 3 re-implemented the same model as a single-threaded, deterministic C program and re-checked the safety invariants at run time.

The geometry and the contract are unchanged. The main street runs **EAST–WEST** and the side street **NORTH–SOUTH**, with the RSU owning the conflict zone. M/M/1 lets one vehicle cross at a time; M/M/2 lets two cross if they do not conflict, selected by `MM_MODE`. Priority is *emergency* → *main street* → *side street*, and a right turn is taken before a left turn inside a conflict. The critical pair is **V4** (North, right turn) and **V5** (South, left turn), which must never cross together; in UPPAAL this is `A[] not (crossing[4] && crossing[5])`. Table 1 lists the exact fleet that was verified.

Table 1. The verified six-vehicle fleet (as instantiated in the UPPAAL `system` block).

Veh.	Origin	Turn	Emerg.	Period	Cross	Role
V0	EAST	STRAIGHT	yes	30	4	Emergency vehicle (main street)
V1	EAST	STRAIGHT	no	6	5	Main-street, used for the M/M/2 overlap
V2	WEST	STRAIGHT	no	8	5	Main-street, opposite direction
V3	NORTH	STRAIGHT	no	10	5	Side-street straight
V4	NORTH	RIGHT	no	12	5	Critical right-turn vehicle
V5	SOUTH	LEFT	no	12	5	Critical left-turn vehicle

Week 3 also surfaced one problem that this proposal must address. Under strict static priority with continuous arrivals, **V5 is never served**, even when the run is extended to 400 ticks. This is not a modelling bug: the UPPAAL queries check safety and reachability, not liveness, so nothing forbids it. With the priority encoding in the model, `priorityValue` gives $V0=0 < V1=V2=11 < V4=20 < V3=21 < V5=22$, so V5 sits at the strict bottom of every tie-break and can be perpetually overtaken. Section 8 fixes this with an aging term in software.

3 Codesign Methodology and Partitioning Criterion

We follow the standard codesign flow from the module: start from the reference implementation, sort each function by *how it computes*, then place the work accordingly. The criterion is a single rule:

Regular, parallel and timing-critical work → hardware. Irregular, control-heavy and often-changed work → software.

Applied to the Week 3 functions, the split is clear-cut.

3.1 Functions that stay in software

The scheduler loop, the V2I/I2V/V2V message handling, and the state updates `requestVehicle()`, `grantVehicle()` and `finishVehicle()` are sequential, branch-heavy, and tied to the per-vehicle timers for `period` and `crossTime`. The priority functions `priorityValue()`, `betterPriority()`, `existsBetterGrantable()` and `emergencyWaiting()` decide the order of service and are exactly the parts we are most likely to change later — for example when adding aging. All of this maps naturally onto FreeRTOS tasks and queues and stays in software.

3.2 Functions that move to hardware

The conflict checking is different in kind. `conflict(a,b)`, `conflictsWithCrossing(i)` and `noCollision()` are pure functions of the current `origin[]`, `turn[]`, `emergency[]` and `crossing[]` values. They hold no state of their own and use no timers. In software they are also the most repeated work on the decision path:

- `noCollision()` is an $O(N^2)$ double loop — 15 pairwise `conflict()` checks for $N = 6$ — and it runs on every tick as the safety monitor.
- `conflictsWithCrossing(i)` is an $O(N)$ scan, and the capacity test calls it for every candidate the RSU examines during a grant.

In hardware these 15 pairwise checks all evaluate in parallel in a single clock cycle, so the timing is constant and easy to bound, and the CPU stops spending cycles on them. The same combinational block keeps running every clock against the live `crossing[]` set, giving an independent hardware check of the `A[] noCollision()` invariant. That dual benefit — it helps both *timing* and *safety* — is the main reason this part belongs in hardware. Figure 1 shows the proposed partition, and Figure 2 shows why this kernel dominates a single grant decision.

4 Proposed System Architecture

4.1 Software part (FreeRTOS)

- **Vehicle tasks (V0–V5):** one task each, running the Vehicle FSM from UPPAAL (`Idle`, `WaitingInQueue`, `V2V_Enter`, `Crossing`, `V2V_Leave`). The clock `x[i]` becomes a FreeRTOS software

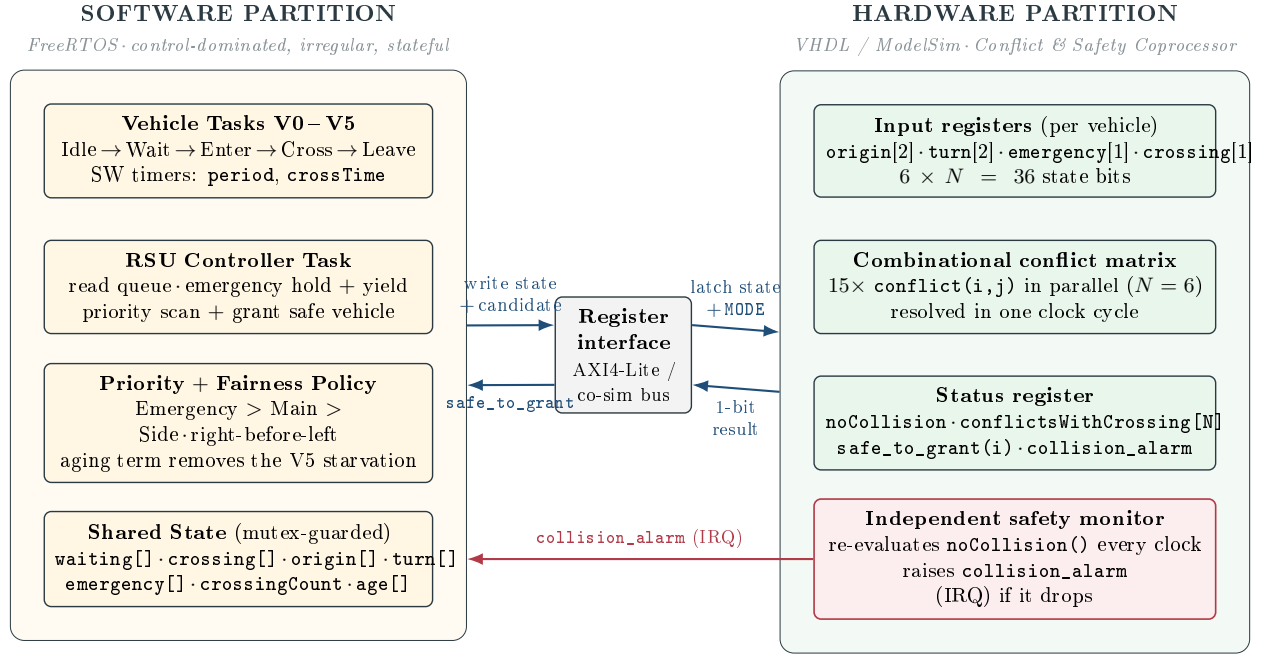


Figure 1. Proposed HW/SW codesign partition. The software side (FreeRTOS) keeps the admission and scheduling logic, the messages and the queue state; the hardware side (VHDL/ModelSim) holds the *Conflict & Safety Coprocessor*, which returns one `safe_to_grant` bit per candidate and runs an independent collision monitor wired to an interrupt.

timer / `vTaskDelay` for `period` and `crossTime`; the committed enter and leave steps remain zero-delay transitions.

- **RSU controller task:** reads the requests, runs the priority scan and the grant loop, sends the grants and the emergency yield, and owns the queue state. Instead of computing the conflicts itself, it queries the coprocessor.
- **Communication:** the channels `req[]`, `emg_req[]`, `grant[]`, `leave[]`, `i2v_yield`, `v2v_enter` and `v2v_leave` become FreeRTOS message queues; the broadcast channels are handled as one-to-many task notifications.
- **Shared state:** `waiting[N]`, `crossing[N]`, `origin[N]`, `turn[N]`, `emergency[N]`, `crossingCount` and `emergencyNotified`, guarded by a mutex and mirrored into the coprocessor input registers immediately before each decision.

4.2 Hardware part (VHDL / ModelSim): the Conflict & Safety Coprocessor

This is a combinational module with a small register interface that resolves the conflict relation for the whole fleet at once.

- **Inputs (registered):** `origin` (2 bits), `turn` (2 bits), `emergency` (1 bit) and `crossing` (1 bit) per vehicle — $6 \text{ bits} \times N = 36 \text{ bits}$ of state.
- **Conflict matrix:** 15 `conflict(i,j)` cells in parallel, each a small Boolean expression over `isMainRoad`, `oppositeDirection`, the same-origin case and the turn rules. This is a direct translation of the C `conflict()` function into logic.
- **Outputs (status register):** `noCollision` (1 bit), `conflictsWithCrossing[N]` (N bits), the capacity/`safe_to_grant(i)` bit for the active `MM_MODE`, and `collision_alarm`.
- **Safety monitor:** the same matrix is evaluated every clock against the live `crossing[]` set. If `noCollision` ever goes low it raises `collision_alarm` as an interrupt — an independent check that matches the ISO 26262 idea of freedom from interference.

5 The `safe_to_grant` Signal

In Week 3, `canGrant(i)` checks four conditions in sequence. For the codesign we keep `canGrant()` in software but move only the conflict-related check into hardware, so the function still reads clearly:

```

bool canGrant(int i) {
    if (!waiting[i])                                return false; // SW: queue
                                                    state
    if (emergencyWaiting() && emergency[i]==0) return false; // SW:
                                                    emergency hold
    if (!hw_safe_to_grant(i))                      return false; // HW:
                                                    capacity + conflict (1 clock)
    if (existsBetterGrantable(i))                  return false; // SW: strict
                                                    priority (+ aging)
    return true;
}

```

Here `hw_safe_to_grant(i)` replaces the Week 3 `capacityAvailableFor(i)`, which used to call `conflictsWithCrossing(i)`. In M/M/1 it is true only when `crossingCount` is 0. In M/M/2 it is true when fewer than two vehicles are crossing *and* candidate *i* conflicts with none of them — and that conflict part is exactly what the hardware computes in parallel. The answer is one bit, which turns the most expensive step of the decision into a single register read.

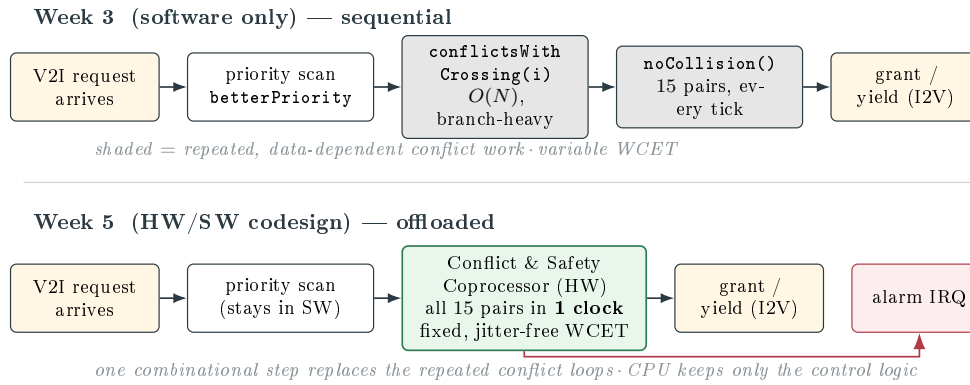


Figure 2. One grant decision. The repeated $O(N)$ and $O(N^2)$ conflict evaluation from Week 3 (shaded) collapses into a single combinational hardware step in Week 5; the software keeps only the irregular control logic, and the coprocessor additionally drives the collision-alarm interrupt.

6 HW/SW Interface

Software and hardware communicate through a small set of memory-mapped registers. On a real Zynq or Kria board this is an AXI4-Lite slave; in the ModelSim co-simulation it is a simple register bus. The interface is kept deliberately minimal: *write the state, latch, read the result*. Because the conflict matrix is combinational, the answer is valid in the same bus access. `ST_ALARM` is wired to an interrupt, so a safety violation is caught on its own, separately from the scheduling code.

Table 2. Memory-mapped register interface between the FreeRTOS RSU task and the coprocessor.

Register	Dir	Width	Meaning
STATE_VEH[i]	W	6 bits	origin(2), turn(2), emergency(1), crossing(1) for vehicle <i>i</i>
MODE	W	1 bit	MM_MODE: 0 = M/M/1, 1 = M/M/2
CAND	W	3 bits	candidate id to test for <code>safe_to_grant</code>
CTRL	W	1 bit	start / latch inputs (result valid in the same access)
ST_SAFE	R	1 bit	<code>safe_to_grant(CAND)</code> for the current state and MODE
ST_NOCOL	R	1 bit	<code>noCollision()</code> over the whole fleet
ST_CWC	R	<i>N</i> bits	<code>conflictsWithCrossing[i]</code> bit-vector
ST_ALARM	R/IRQ	1 bit	<code>collision_alarm</code> (latched safety violation)

For each decision the RSU task writes the current `STATE_VEH[]` and `MODE` once, then for each candidate writes `CAND` and reads `ST_SAFE`. A single writer (the RSU task) mirroring state before

each decision keeps hardware and software from drifting.

7 Construct-by-Construct Mapping

Each part of the verified model has exactly one execution target, which preserves the well-defined mapping from models that the brief requires.

Table 3. HW/SW assignment of every model and code construct, with the reason.

UPPAAL / C construct	Target	Reason
<code>conflict(a,b)</code>	HW	pure Boolean of static rules; 15 cells in parallel
<code>conflictsWithCrossing(i)</code>	HW	$O(N)$ scan becomes an N -bit combinational vector
<code>noCollision()</code>	HW	$O(N^2)$ monitor every tick; one cycle plus alarm IRQ
<code>capacityAvailableFor(i)</code>	HW + SW	capacity compare folded into <code>safe_to_grant</code>
<code>priorityValue</code> / <code>betterPriority</code>	SW	orchestration; changes with the aging term
<code>existsBetterGrantable(i)</code>	SW	depends on the priority policy, kept flexible
<code>emergencyWaiting()</code>	SW	trivial scan; drives the I2V yield broadcast
<code>canGrant(i)</code>	SW	thin wrapper that calls <code>hw_safe_to_grant()</code>
<code>requestVehicle</code> / <code>grantVehicle</code> / <code>finishVehicle</code>	SW	state updates plus queue and message side effects
Vehicle FSM (5 locations) + clock <code>x[i]</code>	SW	one FreeRTOS task per vehicle, with software timers
<code>RSU_Controller</code> (req/- <code>grant/leave/yield</code>)	SW	RSU task plus FreeRTOS queues
<code>channels</code> <code>req/grant/leave/i2v_yield/v2v_*</code>	SW	message queues / task notifications

8 Extension: Fairness / Aging (fixing the V5 starvation)

To stop V5 from starving, we add an aging term so that the longer a vehicle waits, the higher its effective priority becomes. A per-vehicle counter `age[i]` increments every tick while `waiting[i]` is true and resets when the vehicle is granted, and the comparison in `betterPriority()` becomes

$$\text{effectivePriority}(i) = \text{priorityValue}(i) - k \cdot \text{age}[i].$$

Aging is small, stateful and policy-dependent, so it belongs in software inside `betterPriority()`; the hardware conflict check is untouched. Crucially, **aging never overrides safety**: emergency exclusivity, the capacity limit and the V4/V5 mutual exclusion are still enforced by `safe_to_grant`, so aging only re-orders which of the *already-safe* candidates is chosen. This raises the Week 3 guarantee from *safety only* to *bounded waiting*, which we add as a new liveness query in Section 9.

9 Verification and Co-Simulation Plan (Week 5)

The plan keeps the verified UPPAAL queries as the contract and checks both sides against them.

- **Hardware (ModelSim)**: a VHDL testbench feeds the coprocessor with state vectors and compares `noCollision` and `safe_to_grant` against reference values taken from the Week 3 C run. The key vector is V4 (North-right) and V5 (South-left) both crossing, which must report a conflict.
- **Exhaustive check**: the input space is small (6 bits per vehicle over the few origin and turn values the fleet actually uses), so the hardware outputs can be compared against the C reference over *all* reachable states, not just a sample.
- **Software (FreeRTOS)**: the FreeRTOS version must reproduce the Week 3 trace — the M/M/2 overlap of V1 and V2, and the emergency case where V0 pre-empts after an I2V yield — with the three safety invariants kept as run-time assertions.

- **Co-simulation:** the RSU task drives the coprocessor over the register interface; the test fails if `hw_safe_to_grant` ever disagrees with the C reference.

Table 4. Verification correspondence for the codesign, with the new liveness query added by aging.

UPPAAL query	Checked where	Mechanism
<code>A[] not deadlock</code>	SW	cyclic schedule always has an enabled action
<code>A[] noCollision()</code>	HW + SW	HW monitor every clock, plus SW assertion
<code>A[] crossingCount <= MM_MODE</code>	HW + SW	capacity in <code>safe_to_grant</code> , plus SW assertion
<code>A[] not(crossing[4] && crossing[5])</code>	HW + SW	conflict cell $(4, 5) = 1$, used as a regression vector
<code>E<> crossing[0]</code>	SW	emergency V0 is granted (log + counter)
<code>E<> crossing[1] && crossing[2]</code>	SW	V1 and V2 cross together in M/M/2
<code>waiting[5] -> crossing[5] (new)</code>	SW	bounded waiting once aging is enabled

10 Week 5 Work Plan, Deliverables and Risks

10.1 Deliverables

- **SW:** the FreeRTOS implementation (vehicle tasks, RSU task, queues, timers, aging) reproducing the Week 3 behaviour.
- **HW:** a synthesisable VHDL Conflict & Safety Coprocessor with a ModelSim testbench and the golden regression vectors.
- **Integration:** the register-interface driver plus a co-simulation demonstration, pushed to the repository.
- **Report:** a chapter documenting the partition, the interface and the verification results.

10.2 Risks and mitigations

Risk	Mitigation
HW/SW state drift (stale registers)	one writer (the RSU task) mirrors the state before each decision; a mutex guards the shared arrays
Interface overhead cancels the HW benefit	keep it combinational with a single-cycle register read; write the state once per tick
ModelSim setup or licensing	use the Intel/Quartus Prime ModelSim edition from the brief and start the testbench early
Aging breaks a safety invariant	aging only re-orders <i>safe</i> candidates; safety stays gated by <code>safe_to_grant</code> ; add the liveness query
Behaviour drifts from Week 3	golden vectors and assertions tie both parts back to the verified C reference

11 Summary

The proposed split preserves the verified contract while placing each part of the Week 3 core where it fits best: the irregular, frequently-changed scheduling and messaging logic in software on FreeRTOS, and the regular, parallel, safety-relevant conflict checking (`conflict`, `conflictsWithCrossing`, `noCollision`) in a hardware *Conflict & Safety Coprocessor* that returns one `safe_to_grant` bit and runs an independent collision monitor. This yields a grant decision with constant, bounded timing, a second hardware check of `A[] noCollision()`, and a lighter CPU load, while the aging term lifts the guarantee from *safety only* to *bounded waiting*. Week 5 is therefore a focused implementation and verification task in FreeRTOS and ModelSim.